

GESTIÓN DE DATOS DE PAÍSES EN PYTHON

TRABAJO INTEGRADOR – PROGRAMACIÓN I

Alumnos:

Jesica Belén Molina
Facundo Isaac Avellaneda

Fecha:

17 de junio de 2026

ÍNDICE

Marco Teórico e Investigación	Pág. 3
Decisiones Técnicas y Arquitectura de Software	Pág. 5
Dificultades y Conclusiones	Pág. 9
Bibliografía y Fuentes Consultadas	Pág. 10
Accesos al Proyecto (Repositorio y Video)	Pág. 11

MARCO TEÓRICO E INVESTIGACIÓN

- **Listas:** son estructuras de datos mutables que permiten almacenar una secuencia ordenada de elementos, los cuales pueden ser de cualquier tipo. En el software desarrollado, la lista es la estructura que almacena los diccionarios de cada país. Su función es centralizar todos los registros cargados desde el archivo CSV para que el programa pueda recorrerlos, filtrarlos, ordenarlos o modificarlos en tiempo real antes de guardarlos definitivamente.
- **Diccionarios:** son estructuras de datos que permiten almacenar información mediante un sistema de pares clave-valor. Es decir, cada elemento que contiene tiene una clave (key) única, asociada a un valor (value). Sus principales características son:
 - Cada clave es única dentro del diccionario. No puede haber dos claves iguales.
 - Los valores pueden ser de cualquier tipo de dato: números, cadenas, listas, otros diccionarios, etc.
 - Los diccionarios son mutables: se pueden agregar, modificar o eliminar pares clave-valor después de su creación.
 - Son no ordenados hasta versiones anteriores de Python 3.6. A partir de Python 3.7 en adelante, los diccionarios mantienen el orden de inserción, aunque no deben considerarse estructuras ordenadas en el mismo sentido que las listas.

Se utilizaron para modelar la entidad "País", donde cada variable (nombre, continente, población, superficie, PBI) se almacena bajo una clave específica, facilitando un acceso asociativo rápido.

- **Funciones:** bloques de código reutilizables que realizan una tarea específica. Principales beneficios de su uso:
 - Reutilización: Podemos llamar la misma función en diferentes partes del código.
 - Modularidad: Se pueden dividir programas complejos en funciones más pequeñas y manejables.
 - Claridad: Hace que el código sea más fácil de entender y mantener.
 - Mantenimiento: Si hay un error en una función, solo se corrige en un solo lugar.
 - Evita redundancia: Se evita escribir el mismo código varias veces.

Estructuramos nuestro sistema en funciones independientes y aisladas para cada operación del menú (tales como la carga del archivo CSV, el alta de registros, la modificación de datos y el algoritmo de ordenamiento) para evitar la redundancia y mejorar el mantenimiento.

- **Condicionales:** Estructuras de control (if-elif-else) que permiten ejecutar diferentes bloques de código según si una condición se cumple o no. Esto es fundamental para la toma de decisiones en un programa. Todos los elementos de un bloque deben tener la misma indentación. La estructura **if** evalúa una condición y, si es True, ejecuta el bloque de código correspondiente. Usamos **else** cuando necesitamos que una acción se ejecute si la condición es False. Cuando hay más de dos posibles escenarios, usamos **elif** (abreviatura de else if).

En nuestro programa, constituyen el núcleo lógico para la navegación de los submenús y operan como una barrera de seguridad en las validaciones, impidiendo el procesamiento de campos vacíos o tipos de datos incorrectos que puedan provocar fallas en el sistema.

- **Ordenamientos:** Ordenar es reacomodar los elementos de una colección según algún criterio. El ordenamiento cumple una función fundamental ya que:
 - Facilita la búsqueda eficiente (como en la búsqueda binaria).
 - Permite organizar datos para visualizaciones o reportes.
 - Es la base de muchos algoritmos y estructuras más avanzadas.

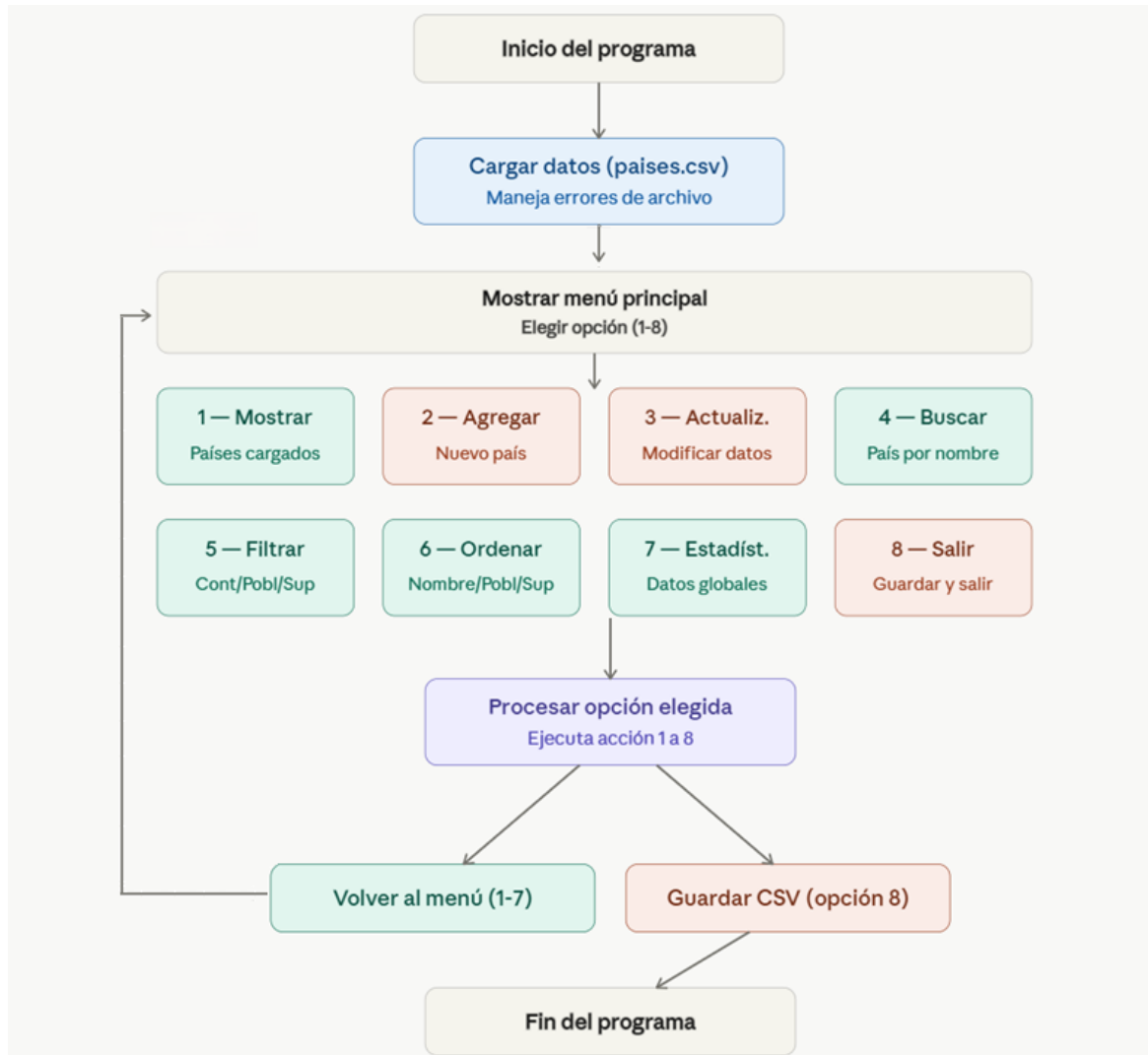
En el software desarrollado, se optó por utilizar el enfoque nativo de Python mediante la función integrada **sorted()**. Esta función permite ordenar de forma eficiente la lista de países a través de funciones clave de extracción (key) para clasificar según Nombre, Población o Superficie, tanto de forma ascendente como descendente mediante el parámetro reverse.

- **Estadísticas Básicas:** Operaciones matemáticas descriptivas orientadas al análisis de conjuntos numéricos. Se aplicaron para calcular máximos, mínimos y promedios aritméticos sobre las variables de población y superficie del dataset.
- **Archivos CSV:** un archivo es un conjunto de datos almacenado en el disco. Los CSV son similares a planillas: cada línea representa una fila, y los valores están separados por comas.

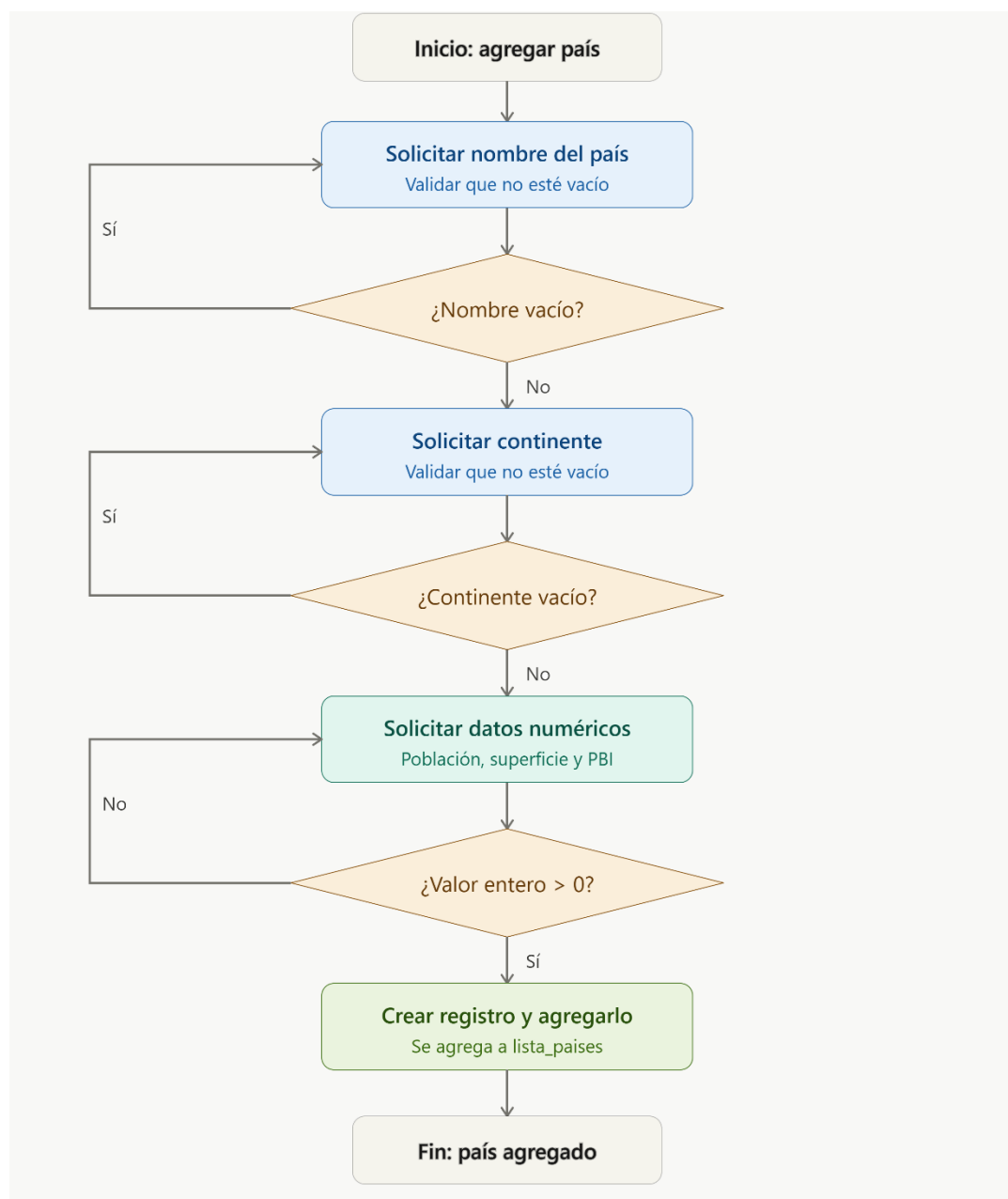
El programa implementa procesos robustos de lectura y escritura para asegurar que las modificaciones en memoria se conserven en el almacenamiento físico.

DECISIONES TÉCNICAS Y ARQUITECTURA

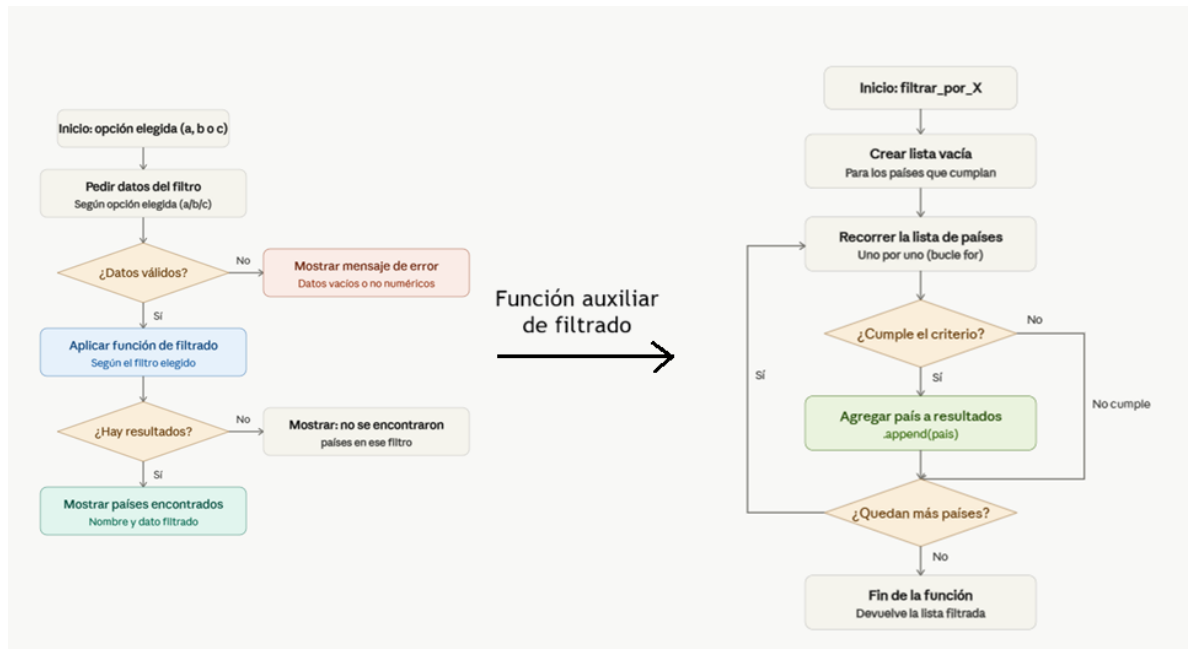
- Diagrama de Flujo del programa:



- Diagrama de Flujo de Agregar país:



- Diagrama de filtrar países



- Capturas de Pantalla de la Ejecución
 - Menú del programa en ejecución

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE

PS C:\Users\belu6\OneDrive\Escritorio\Tecnatura Programacion UTN\Program
n3.11.exe "c:/Users/belu6/OneDrive/Escritorio/Tecnatura Programacion UTN
Archivo 'países.csv' creado con éxito.

=====
          SISTEMA DE GESTIÓN DE DATOS DE PAÍSES
=====
1. Mostrar todos los países cargados
2. Agregar un nuevo país (Alta)
3. Actualizar datos de un país (Modificación)
4. Buscar un país por nombre
5. Filtrar países (Continente / Población / Superficie)
6. Ordenar países (Nombre / Población / Superficie)
7. Mostrar estadísticas globales
8. Salir y guardar
=====
Elegí una opción (1-8): █
    
```

- Manejo de errores

```
=====
                        SISTEMA DE GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países cargados
2. Agregar un nuevo país (Alta)
3. Actualizar datos de un país (Modificación)
4. Buscar un país por nombre
5. Filtrar países (Continente / Población / Superficie)
6. Ordenar países (Nombre / Población / Superficie)
7. Mostrar estadísticas globales
8. Salir y guardar
=====

Elegí una opción (1-8): 2

--- REGISTRAR NUEVO PAÍS ---
Ingresá el nombre del país:
El nombre no puede estar vacío.
Ingresá el nombre del país: Argentina
Ingresá el continente: America
Ingresá la población (solo números): uruguay
Ingreso inválido. Debe ser un número entero mayor a 0.
Ingresá la población (solo números):
```

- Prueba de filtros

```
=====
                        SISTEMA DE GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países cargados
2. Agregar un nuevo país (Alta)
3. Actualizar datos de un país (Modificación)
4. Buscar un país por nombre
5. Filtrar países (Continente / Población / Superficie)
6. Ordenar países (Nombre / Población / Superficie)
7. Mostrar estadísticas globales
8. Salir y guardar
=====

Elegí una opción (1-8): 5

--- SUBMENÚ DE FILTROS ---
a. Filtrar por Continente
b. Filtrar por Rango de Población
c. Filtrar por Rango de Superficie
Elegí una opción de filtrado (a, b o c): b

-- Rango de Población --
Ingresá la población MÍNIMA: 12000
Ingresá la población MÁXIMA: 470000000

Países dentro del rango de población (12000 - 470000000):
-> Argentina (Población: 46000000)
-> Brasil (Población: 214000000)
-> España (Población: 47000000)
-> Japón (Población: 125000000)
-> Egipto (Población: 109000000)
```


DIFICULTADES Y CONCLUSIONES

- Dificultades Técnicas y Soluciones: El mayor desafío técnico residió en la **persistencia de datos** y el control de tipos. Inicialmente, al leer el archivo CSV, todas las variables se interpretaban como cadenas de texto (str), lo que provocaba errores lógicos al intentar realizar ordenamientos numéricos o cálculos estadísticos. Este obstáculo se superó implementando una conversión explícita de tipos (int) acompañada de un riguroso manejo de excepciones y validaciones con `.isdigit()` para asegurar la robustez del software ante entradas inválidas del usuario.
- Conclusiones y Aprendizajes Grupales: La realización de este proyecto integrador nos permitió consolidar conceptos teóricos a través de su aplicación práctica, fortaleciendo especialmente nuestra comprensión sobre la importancia de la **modularización** en el desarrollo de software. Dividir la aplicación en funciones con responsabilidades específicas no solo facilitó el trabajo colaborativo mediante el uso de ramas de Git, sino que también contribuyó significativamente a mejorar la legibilidad, el mantenimiento y la escalabilidad del código.
Por otro lado, trabajar con el pasaje de información entre estructuras dinámicas en memoria (listas de diccionarios) y su persistencia en archivos físicos (formato CSV) nos brindó una visión concreta y valiosa sobre el ciclo de vida de los datos en una aplicación real, desde su creación y manipulación en tiempo de ejecución hasta su almacenamiento permanente.
En síntesis, este trabajo integrador no solo reforzó nuestros conocimientos técnicos en Python, sino que también potenció habilidades de trabajo en equipo, organización y resolución colaborativa de problemas.

BIBLIOGRAFÍA

- Actividad 2 – Listas.pdf, Bibliografía de la materia Programación I de la carrera Tecnicatura en Programación a distancia.
- Listas – Actividad 5.pdf, Bibliografía de la materia Programación I de la carrera Tecnicatura en Programación a distancia.
- 5 – Funciones.docx.pdf, Bibliografía de la materia Programación I de la carrera Tecnicatura en Programación a distancia.
- Condicionales.pdf, Bibliografía de la materia Programación I de la carrera Tecnicatura en Programación a distancia.
- Apunte – Archivos.pdf, Bibliografía de la materia Programación I de la carrera Tecnicatura en Programación a distancia.

ENLACES PÚBLICOS DEL PROYECTO

- Link al Repositorio Git: <https://github.com/belu674/TPI-Paises-Programacion1>
- Link al Video Tutorial: <https://youtu.be/4B7cgko6jmU>